TemperaturWatch

Administrator Manual

Waveshare ESP32-P4-WIFI6-POE-ETH

Firmware version 1.1.0

October 2026

Deutsche Version: admin_handbuch_de.pdf

Table of contents

1. Introduction

2. Commissioning

3. Web interface overview

4. Dashboard

5. Sensors

6. Digital IOs

7. Network

8. MQTT

9. SNMP and monitoring

10. System

11. Updates (OTA)

12. Security and operation

13. Troubleshooting

14. Technical data

1. Introduction

TemperaturWatch is a temperature and humidity gateway based on the Waveshare ESP32-P4-WIFI6-POE-ETH. It reads Dallas DS18B20, DHT11, and AM2301 sensors, switches digital outputs (e.g. relays for fans or alarm devices) manually or automatically via threshold rules, and provides all values through a web interface, REST API, MQTT, and SNMP. This manual is written for **administrators** who install the device, integrate it into the network and monitoring environment, and operate it.

> **Notice on use**
> TemperaturWatch is a monitoring and convenience device and **not a safety-certified protection system**. Wherever a failure could cause injury or significant damage (e.g. over-temperature protection), an independent protection device must be provided. Selecting and protecting the connected loads is the operator's responsibility.

Feature overview

Area	Function
Sensors	Dallas DS18B20 (1-Wire, several sensors on one pin), DHT11, AM2301/DHT21/DHT22; up to 32 sensors
Digital IOs	Outputs and inputs, optionally inverted; outputs manually or via threshold rule with hysteresis; up to 32 entries
Network	Ethernet (always active) and WiFi (via the ESP32-C6 co-processor), DHCP or static IP per interface, mDNS hostname
Interfaces	Web interface, REST API, MQTT (optional TLS), SNMPv1/v2c agent
Time	NTP synchronization with selectable timezone
Maintenance	Firmware and web UI update over the network (OTA) with automatic rollback, optional login
Languages	Web interface and help in German and English, day/night mode

Hardware overview

The board carries an ESP32-P4 (32 MB flash, 32 MB PSRAM). WiFi/Bluetooth is provided by an onboard ESP32-C6 co-processor; Ethernet runs through the internal MAC with an IP101 PHY and works without configuration. Sensors and IOs can **only** be connected to the 28 GPIOs of the 40-pin header; all other GPIOs are used internally (Ethernet, WiFi link, SD card, USB, display, audio) and are rejected by the firmware. All GPIOs operate at 3.3 V.

Pin	Signal	Pin	Signal	Pin	Signal	Pin	Signal
1	5V	11	GPIO22	21	GPIO1	31	GPIO54
2	3V3	12	GPIO21	22	GPIO2	32	GPIO26
3	5V	13	GND	23	GPIO36	33	GND
4	GPIO7 (I2C SDA)	14	GPIO20	24	GPIO0	34	GPIO48
5	GND	15	GPIO5	25	GPIO32	35	GPIO46
6	GPIO8 (I2C SCL)	16	GPIO6	26	GND	36	GPIO53
7	GPIO37 (debug UART TX)	17	GPIO4	27	GPIO25	37	GPIO27
8	GPIO23	18	3V3	28	GPIO24	38	GPIO47
9	GPIO38 (debug UART RX)	19	GND	29	GND	39	GPIO45
10	GND	20	GPIO3	30	GPIO33	40	GND

GPIO7/8 (I2C) and GPIO37/38 (debug UART) remain assignable but are marked as having a default function. In the web UI, GPIOs that are already in use are shown struck through.

2. Commissioning

Flashing the firmware (first installation)

A new board only needs to be flashed once via USB; all further updates run over the network (chapter 11). The five files (`bootloader.bin`, `partition-table.bin`, `ota_data_initial.bin`, `temperaturwatch.bin`, `www.bin`) are available on the GitHub release page. Put all files in one folder, connect the board via USB, and run (adjust the port; use `python -m esptool` instead of `esptool` if necessary):

```
python -m esptool --chip esp32p4 -b 460800 --before default_reset --after hard_reset write_flash \
  --flash_mode dio --flash_size 32MB --flash_freq 40m \
  0x2000 bootloader.bin  0x8000 partition-table.bin  0x10000 ota_data_initial.bin \
  0x20000 temperaturwatch.bin  0x820000 www.bin
```

> **Choose the right port**
> If several ESP32 devices are connected to the computer, verify the target's identity before flashing (`esptool chip_id` or MAC address) so that the wrong device is not overwritten.

Finding the device on the network

1. Connect the Ethernet cable (the port supports PoE; alternatively power the board by other means). The device obtains its address via **DHCP**.
2. The web interface is reachable at `http://temperaturwatch.local/` (mDNS hostname, changeable under "Network"). If mDNS does not work on your network, look up the assigned IP address in the DHCP list of your router/server or read it from the serial log (115200 baud).
3. The web UI is reachable **without a password** from the factory – see chapter 12.

First steps

1. Open the web interface and choose the language at the top right (English is the default).
2. Under **Network**, enter WiFi credentials and a hostname if needed, and optionally assign a static IP (2.4 GHz networks only).
3. Create the sensors under **Sensors**, and outputs/inputs and rules under **IOs**.
4. Set up the connection to your broker or monitoring system under **MQTT** and **SNMP**.
5. Under **System**, check the time/timezone and enable a **login**.

3. Web interface overview

The web interface is built into the firmware and needs no cloud and no additional software. The navigation is on the left (Dashboard, Sensors, IOs, Network, MQTT, SNMP, System, Help); the **status bar** is at the top.

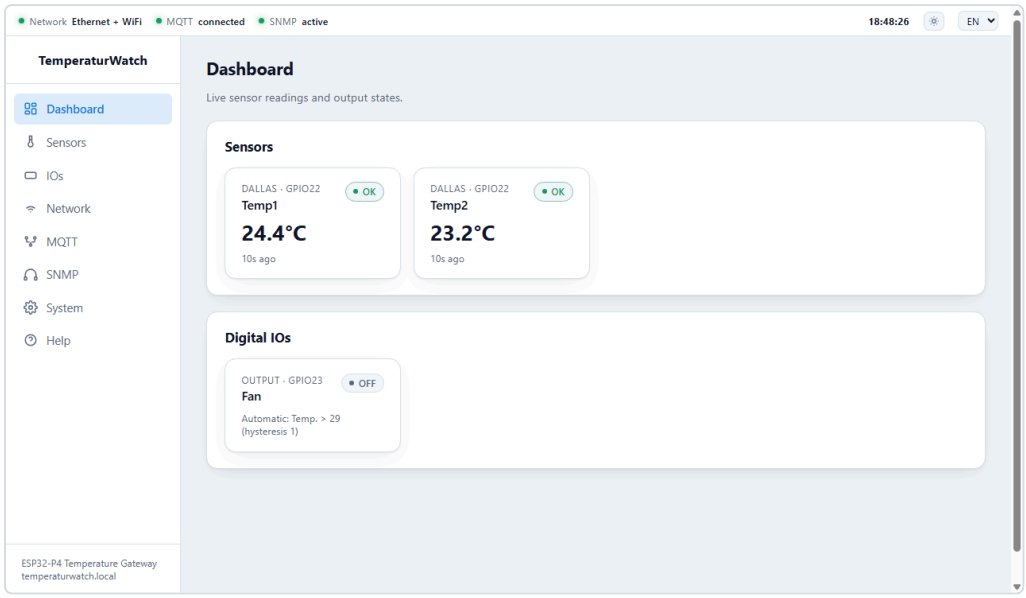


Figure 1: Dashboard with status bar (top) and navigation (left)

Status bar element	Meaning
Network	Shows *Ethernet*, *WiFi*, or *Ethernet + WiFi* as soon as the respective interface has an IP address, otherwise *no IP*
MQTT	*connected*, *disconnected* (enabled but no broker connection), or *disabled*
SNMP	*active*, *restart needed* (enabled but not started yet), or *disabled*
Time	Device time in the configured timezone; shown *dimmed* until synchronized via NTP (tooltip with date)
Sun icon	Day/night mode toggle; the choice is stored in the browser
EN / DE	Language of the web interface and help; the choice is stored in the browser, `?lang=de` in the URL takes precedence

The **Help** page contains the REST API, MQTT, and PRTG guides directly on the device – readable without internet access.

4. Dashboard

The dashboard shows all sensors (type, GPIO, label, reading, age of the reading, status) and all IOs and refreshes every four seconds. Outputs without a rule can be switched on and off directly with a button; for outputs with an active threshold rule, the rule ("Automatic: …") is shown instead and manual switching is blocked.

Sensor status	Meaning
waiting	No reading yet since startup or creation
OK	Last reading succeeded
Error	Last reading failed (wiring, sensor, pull-up – see chapter 13)

5. Sensors

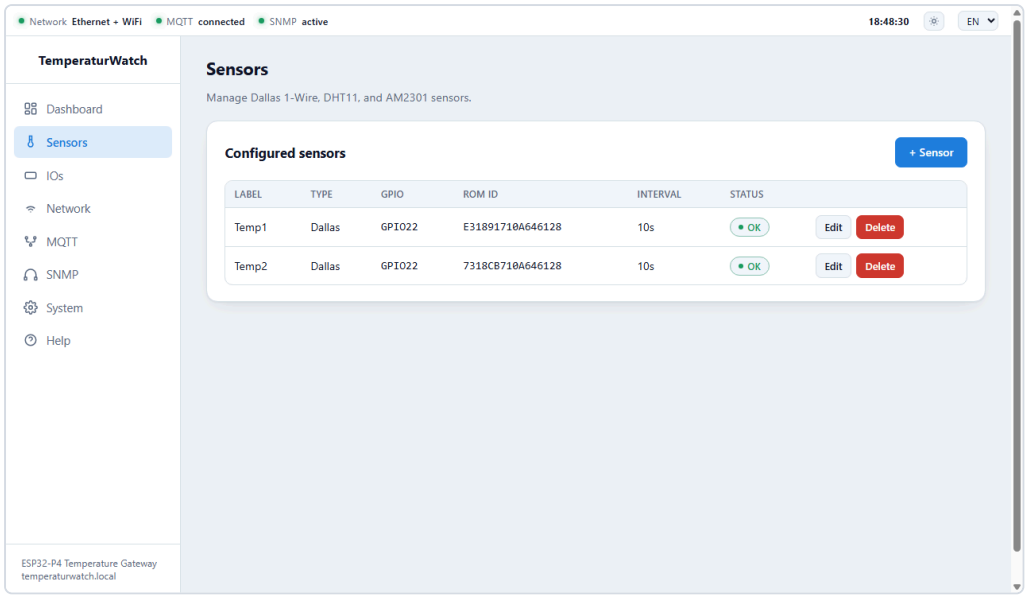


Figure 2: Configured sensors

Type	Notes
Dallas DS18B20	1-Wire bus. **Several Dallas sensors may share the same GPIO** and are told apart by their ROM ID. Provides temperature only.
DHT11	Temperature and humidity, coarse resolution. One sensor per GPIO.
AM2301 / DHT21 / DHT22	Temperature and humidity, higher accuracy. One sensor per GPIO.

Creating a sensor

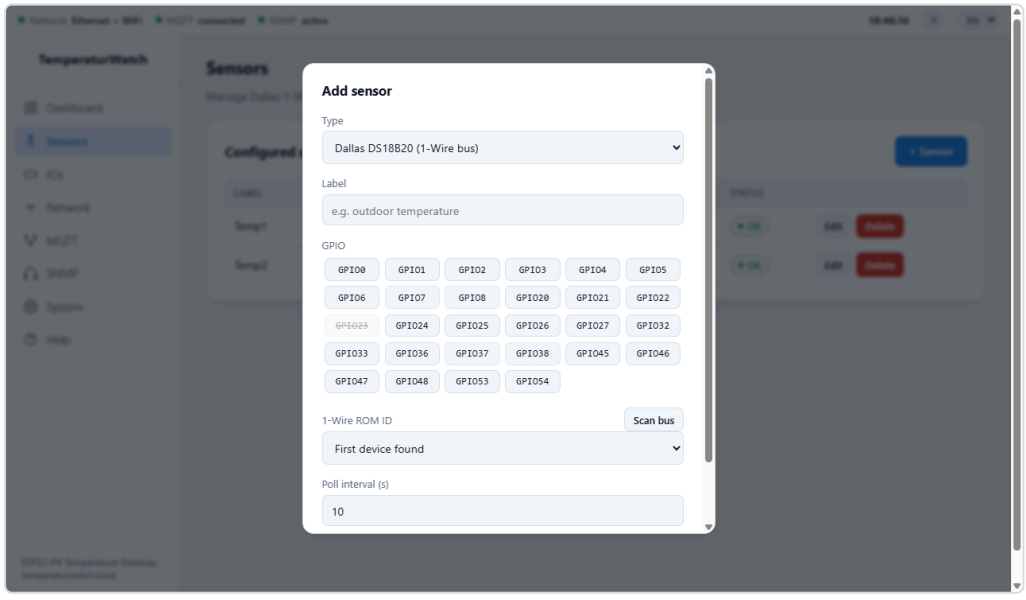


Figure 3: "Add sensor" dialog

1. Choose **+ Sensor**, set the **type**, and enter a **label** (max. 47 characters). The technical ID used in MQTT topics and the REST API is derived from the label.
2. Click the **GPIO** of the header. Struck-through GPIOs are already in use.
3. Dallas only: use **Scan bus** to load the ROM IDs found on the GPIO and pick the desired sensor. Without a selection, the first device found is always read – this only makes sense with *one* sensor on the bus.
4. Set the **poll interval** in seconds (default 10 s, minimum 2 s) and save.

> **Wiring**
> Power sensors with 3.3 V. The data line usually needs a pull-up (typically 4.7 kΩ to 3V3); many sensor modules already include one. For long cables, make sure the ground connection is clean.

6. Digital IOs

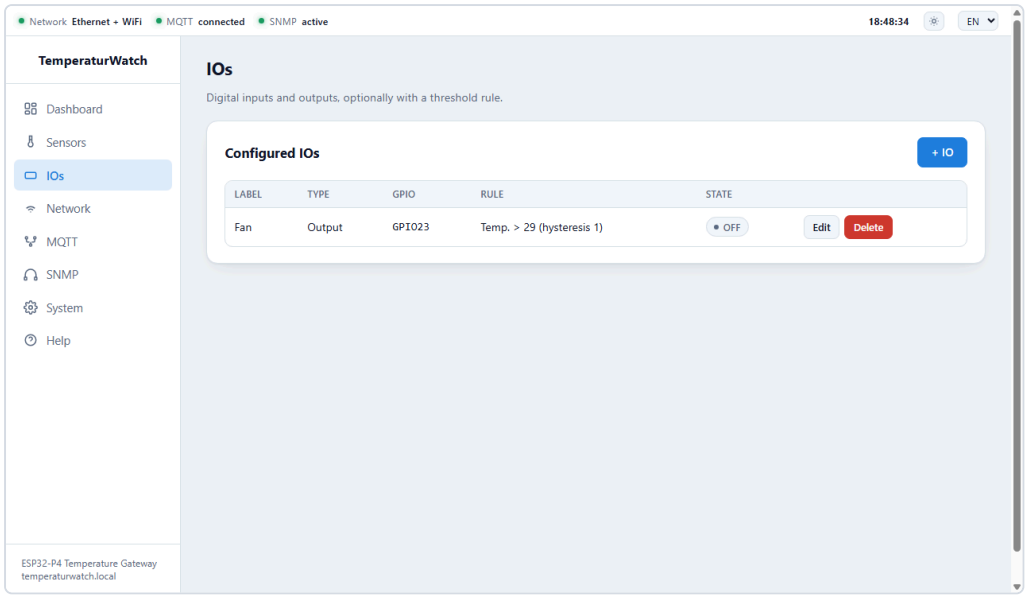


Figure 4: Configured IOs

Setting	Meaning
Type	*Digital output* or *Digital input* (inputs use an internal pull-up, without debouncing)
Inverted (active-low)	Reverses the logic; "ON" then corresponds to a low level on the pin
Initial state: ON	State of an output right after boot
Threshold rule	Switches an output automatically based on a sensor value (see below)

Switching loads

The GPIOs are not designed to drive relay coils or other loads directly. Always drive outputs through a relay module or a driver stage.

Threshold rule with hysteresis

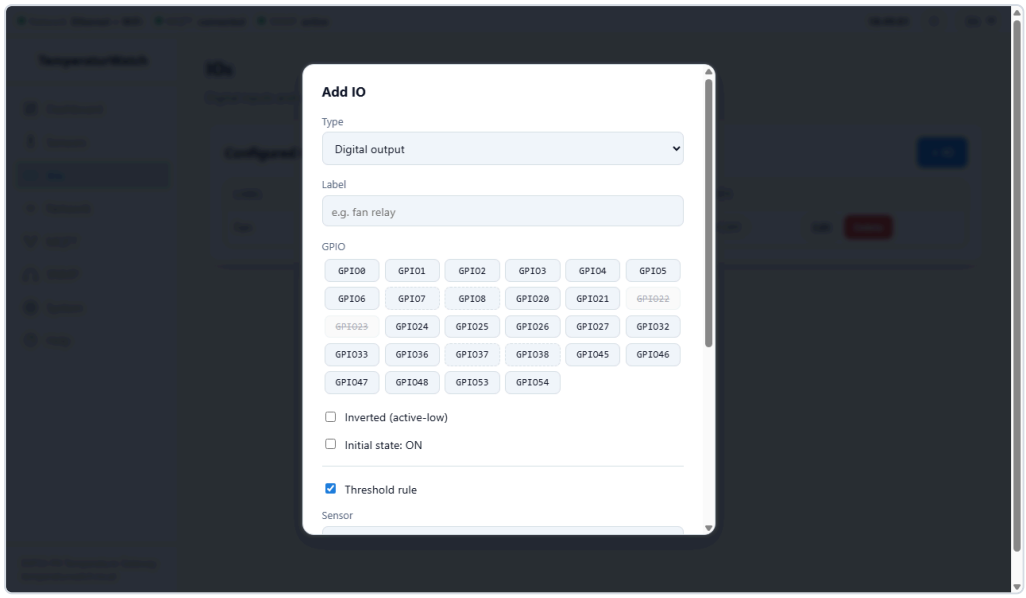


Figure 5: "Add IO" dialog with the threshold rule enabled

A rule links a **sensor** and its **measured value** (temperature or humidity) with a **condition** (>, ≥, <, ≤) and a **threshold**. The **hysteresis** prevents rapid on/off switching (relay chatter) near the threshold:

> **Example: fan relay**
> Condition "> greater than", threshold 25 °C, hysteresis 1 °C: the output switches **on** as soon as the temperature rises above 25 °C and only switches **off** again once it reaches or falls below 24 °C. For "<"/"≤" it works the other way round (off only above threshold + hysteresis). A hysteresis of 0 switches exactly at the threshold.

While a rule is active, it controls the output; manual switching (web UI, REST API, MQTT) is rejected. If the sensor delivers no valid reading, the rule leaves the output unchanged. To switch manually, remove the rule.

7. Network

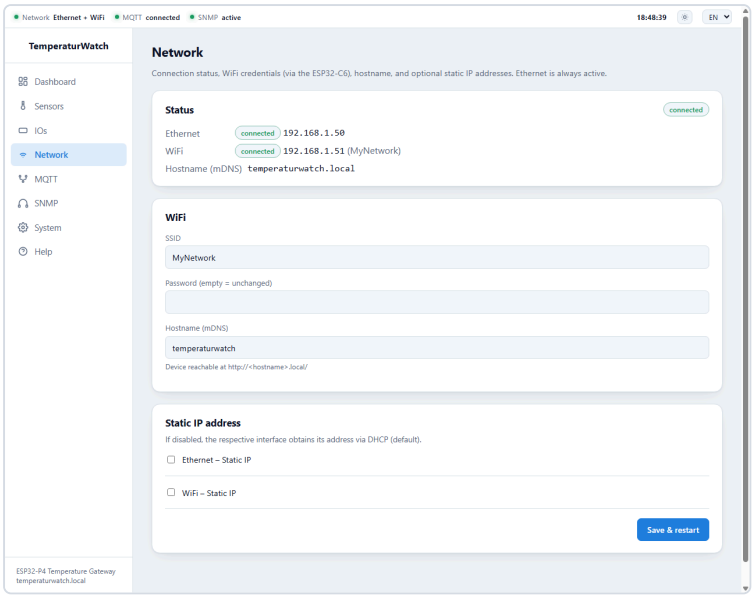


Figure 6: Network – status, WiFi, and static IP

The **Status** panel shows the connection state with the current IP address per interface, as well as the mDNS hostname. Ethernet and WiFi can be active at the same time and then have separate addresses.

Setting	Meaning
SSID / password	WiFi credentials (2.4 GHz only). An empty password field keeps the existing password. Without an SSID, WiFi stays inactive.
Hostname (mDNS)	The device is reachable at `http://<hostname>.local/` (max. 31 characters)
Static IP	Can be enabled separately per interface: IP address, subnet mask, gateway, optional DNS server. Disabled = DHCP.

Save & restart saves the settings and restarts the device – network changes only take effect after the restart.

> **Danger: device no longer reachable**
> A wrong combination of IP address, subnet mask, and gateway makes the device unreachable over the network after the restart (web UI, REST API, MQTT, and SNMP all depend on the same interface). Check the values carefully before saving and never assign the same static IP to Ethernet and WiFi on the same network. Recovery is then only possible via a serial connection (reflashing, see chapter 11).

8. MQTT

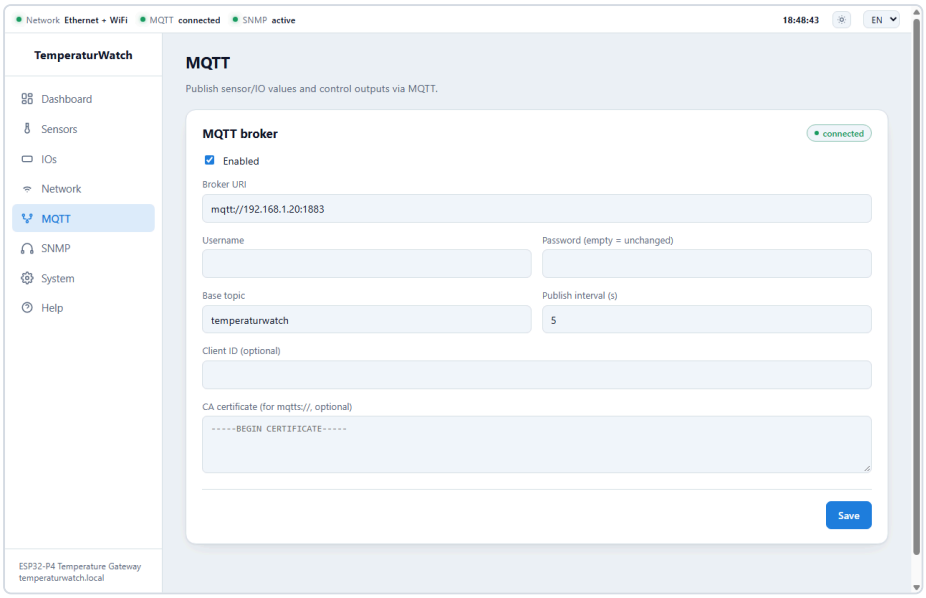


Figure 7: MQTT settings

1. Turn on **Enabled** and enter the **broker URI** (`mqtt://host:1883` or `mqtts://host:8883`).
2. If needed, set username/password (empty password = unchanged), the **base topic** (default `temperaturwatch`), and the publish interval (default 30 s, minimum 5 s). With several devices on one broker, give each device its own base topic.
3. For `mqtts://`, paste the broker's CA certificate (PEM). Client certificates (mTLS) are not supported.
4. Save – the connection is re-established **immediately**, no restart needed. The badge shows the status.

Topic (below base topic)	Direction	Content
`<base>/status`	Device → broker	*online* / *offline* (retained, last will)
`<base>/sensor/<id>/temperature_c`	Device → broker	Temperature, e.g. `21.50`
`<base>/sensor/<id>/humidity_pct`	Device → broker	Humidity (DHT/AM2301 only)
`<base>/io/<id>/state`	Device → broker	*ON* / *OFF* (retained)
`<base>/io/<id>/set`	Broker → device	*ON/OFF/1/0/true/false* switches an output (not while a rule is active)

Details and examples (Home Assistant, Node-RED, TLS) are in the MQTT guide on the Help page and in `docs/MQTT_SETUP.en.md`.

9. SNMP and monitoring

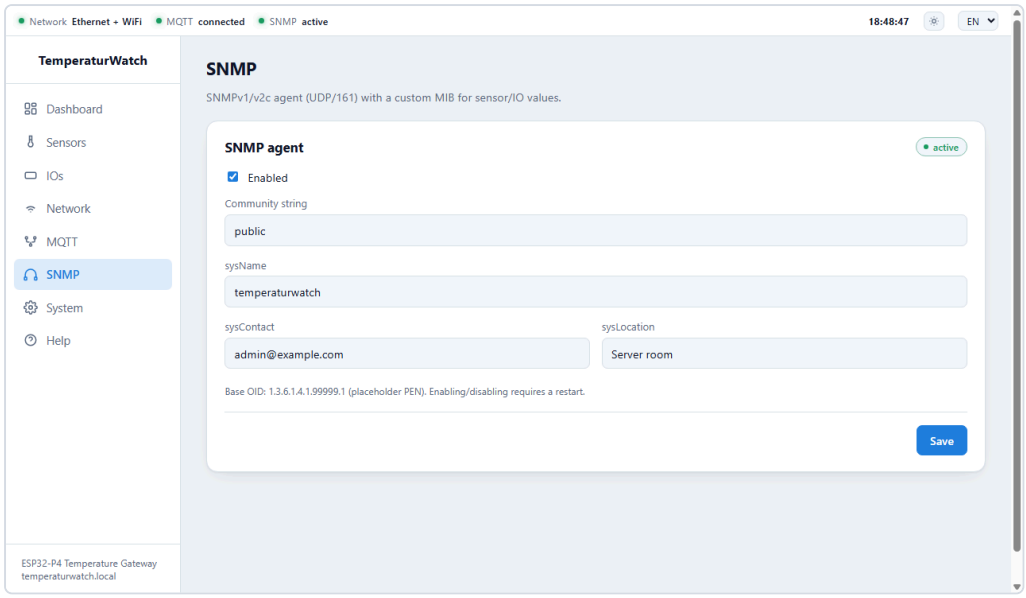


Figure 8: SNMP settings

The SNMPv1/v2c agent listens on UDP port 161. **Enabling or disabling it requires a restart**; changing the community or the system fields (sysName, sysContact, sysLocation) takes effect immediately. The default community is `public` – change it for production use.

OID (base `1.3.6.1.4.1.99999.1`)	Content
`.2.1.0`	sensorCount
`.2.2.1.{1..8}.<n>`	sensorTable: index, ID, label, type, temperature×10, humidity×10 (−1 = n/a), valid, age (s)
`.3.1.0`	ioCount
`.3.2.1.{1..5}.<n>`	ioTable: index, ID, label, type, state
`.1.1.0 … .1.5.0`	System fields (private); additionally the standard MIB-II group `1.3.6.1.2.1.1.*`

> **Table index is not stable**
> The row numbers of the sensor and IO tables result from the order of the configuration. If an entry is deleted from the middle, the following ones move up. Monitoring sensors that query a fixed OID including the row number (e.g. PRTG "SNMP Custom Table") may then show the values of a different sensor. After deleting an entry, check or recreate the monitoring sensors.

Monitoring integration: A step-by-step guide for **PRTG** (no plugin, using "SNMP Custom Table") is on the Help page and in `docs/PRTG_SETUP.en.md`; for **CheckMK** there is a ready-made plugin in `contrib/checkmk/`. Device discovery works because, besides the private MIB, the device also answers the standard MIB-II system group (including sysObjectID).

10. System

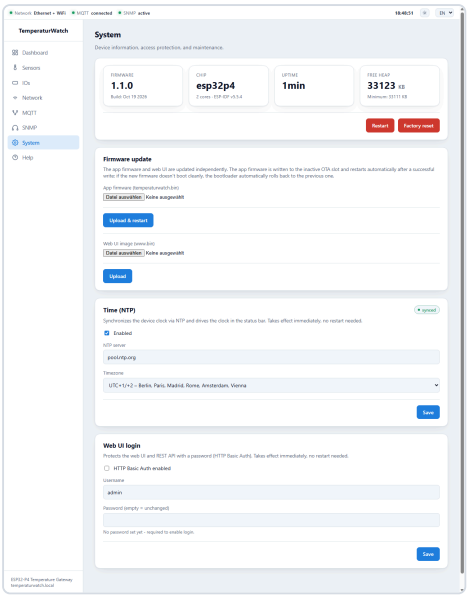


Figure 9: System – information, update, time, and login

Information and restart

The cards show firmware version and build date, chip, uptime, and free heap. **Restart** reboots the device; the configuration is kept. **Factory reset** deletes the network, MQTT, SNMP, sensor, IO, and login configuration and restarts; the time settings are kept. Afterwards the device is reachable without a password again.

Time (NTP)

NTP is active from the factory (server `pool.ntp.org`, timezone Europe/Berlin). The badge shows *synced* or *waiting for sync*. Choose the timezone from a list of common regions, or any other one via "Custom" as a POSIX TZ string (e.g. `CET-1CEST,M3.5.0,M10.5.0/3`). On networks without internet access, enter an internal NTP server; the device needs access to UDP port 123. Changes take effect immediately.

Login

Enables password protection (HTTP Basic Auth) for the web interface *and* the REST API. A password must be set before login can be enabled. Changes take effect immediately; afterwards the browser asks for the credentials.

11. Updates (OTA)

Firmware and web UI are updated independently over the network – in the web UI under **System** → **Firmware update** or via `curl`. No USB cable is needed.

File	Purpose	Behavior
`temperaturwatch.bin`	App firmware	Written to the inactive OTA slot; the device then restarts automatically
`www.bin`	Web UI and help	Written directly to the web partition; **no restart**, the web UI is briefly unavailable meanwhile

```
curl -X POST --data-binary @temperaturwatch.bin http://<ip>/api/system/ota
curl -X POST --data-binary @www.bin             http://<ip>/api/system/ota-web
```

1. Update the **firmware** first and the **web UI** after the restart; always use both from the same release.
2. Reload the browser page (Ctrl+F5) so the new interface is loaded.

> **Automatic rollback**
> If the new firmware does not start cleanly (crash or hang before reaching the main loop), the bootloader automatically returns to the previous version on the next start.

> **Recovery via USB**
> If the web UI is no longer reachable after a failed web UI update, or the device can no longer be reached on the network, the only remedy is reflashing via USB using the command from chapter 2. For a complete fresh start without the old configuration, first run `esptool --chip esp32p4 erase_flash`.

12. Security and operation

- **Enable the login** (System → Login). No password is set from the factory so that you are not locked out after a factory reset.
- The web interface and REST API run over **unencrypted HTTP**; Basic Auth transmits credentials unencrypted. Operate the device only on trusted networks or isolate it with a VLAN/firewall.
- Change the **SNMP community**; SNMPv1/v2c transmits it in clear text and offers no encryption (no SNMPv3).
- Use **MQTT with TLS** (`mqtts://`) and authentication if the broker is outside the local network. The broker must restrict write access to `…/io/+/set` with an ACL – anyone who may publish there can switch outputs.
- Passwords are never returned by the REST API or web UI; an empty password field when saving keeps the existing one. The login password is stored only as a hash; WiFi and MQTT passwords are kept in the device's configuration storage.

Backing up the configuration

There is no export button, but the configurations can be read via the REST API and (for sensors and IOs) restored with `PUT`. Passwords are not included and must be re-entered when restoring.

```
for p in sensors/config io/config network/config mqtt/config snmp/config time/config; do
  curl -s http://<ip>/api/$p -o "backup_$(echo $p | tr / _).json"
done
```

13. Troubleshooting

Symptom	Cause / remedy
Web UI not reachable	Check link LED/cable; look up the IP in the DHCP list or try `temperaturwatch.local`; after a wrong static IP, only reflashing via USB helps
WiFi does not connect	Check SSID/password; 2.4 GHz networks only; was the device restarted after saving? Without an SSID, WiFi stays inactive
Sensor shows "Error" or "waiting"	Check wiring and 3.3 V supply, pull-up on the data line, correct GPIO, poll interval at least 2 s; Dallas: use "Scan bus" to check whether the sensor is found and the ROM ID matches
Dallas bus scan finds nothing	Pull-up (4.7 kΩ to 3V3) missing, data/ground swapped, or wrong GPIO
Output cannot be switched	A threshold rule is active (manual switching blocked) – remove the rule; with inverted logic, "ON" corresponds to a low level
MQTT stays "disconnected"	Check broker URI (`mqtt://`/`mqtts://`), credentials, firewall; for TLS, store the matching CA certificate
SNMP does not respond / "restart needed"	Restart the device after enabling; check community and UDP port 161. Monitoring tools need the standard MIB-II system group for discovery – it is present
Time stays "waiting for sync"	Is the NTP server reachable from the device's network? Allow UDP 123 and DNS, or enter an internal NTP server
Wrong time (daylight saving)	Check the timezone (choose a POSIX TZ string with a daylight saving rule)
Forgot the password	A reset via the interface is not possible (the factory reset is protected as well). Erase with `erase_flash` via USB and reflash – the entire configuration is lost
Old version runs after an update	The new firmware did not report itself as valid and was rolled back automatically – repeat the update, or flash via USB if necessary

14. Technical data

Hardware	Waveshare ESP32-P4-WIFI6-POE-ETH (ESP32-P4 rev. v1.3, 32 MB flash, 32 MB PSRAM, ESP32-C6 as WiFi co-processor)
Network	Ethernet 10/100 (IP101 PHY), WiFi 2.4 GHz; DHCP or static IP per interface; mDNS
Sensor/IO connections	28 GPIOs of the 40-pin header, 3.3 V
Capacity	up to 32 sensors and 32 IO entries; poll interval at least 2 s (default 10 s)
Ports (incoming)	HTTP 80/TCP (web UI, REST API), SNMP 161/UDP
Ports (outgoing)	MQTT 1883/8883 TCP (configurable), NTP 123/UDP, DNS
MQTT	Publish interval from 5 s (default 30 s), last will, optional TLS
SNMP	v1/v2c, private MIB `1.3.6.1.4.1.99999.1` (placeholder PEN) and MIB-II system group
Flash layout	two OTA app slots of 4 MB each, 2 MB web UI partition, configuration storage (NVS)
Software	ESP-IDF v5.5.4, web interface in HTML/CSS/JavaScript without external dependencies
Languages	German / English (web interface and help)

Further documentation

- **REST API reference** (all endpoints with examples): on the Help page and in `docs/REST_API.en.md`
- **MQTT guide**: `docs/MQTT_SETUP.en.md`
- **PRTG guide**: `docs/PRTG_SETUP.en.md`; **CheckMK plugin**: `contrib/checkmk/`
- Source code and releases: github.com/okoebernik/TemperaturWatch

Source code and releases: github.com/okoebernik/TemperaturWatch